

Freight Rate Prediction

How the model was built, what the data turned out to be hiding, and which three approaches were measured and thrown away along the way. Training data 2025-01-01 to 2025-10-31, 48,000 loads; predictions for 12,000 loads across 2025-11-01 to 2025-12-31.

The brief, and the one fact that shaped everything

The task looks like ordinary tabular regression: given a lane, an equipment type, a weight and a date, predict what the load posted for. The wrinkle is in the calendar. Every labelled row falls between 2025-01-01 and 2025-10-31, and every row that gets graded falls in November and December. Nothing is being interpolated. The model is being asked to step two months past the last date it has ever seen.

That single fact drove most of what follows: it is why validation hides future blocks rather than random rows, why no raw time index appears anywhere in the feature set, and why a column that looks like a market signal was worth several hours of attention before it earned its place.

First contact with the data

Forty-eight thousand labelled loads, sixty-four cities, three equipment types, fourteen columns. The first thing worth knowing is how lopsided the signal is: distance correlates with the posted rate at 0.91. A model that only multiplies miles by a constant is already most of the way there, which means the interesting question is never "what does this load cost" but "what does this load cost *per mile*, and why is that number different from the lane's usual one".

So the model predicts log(rate per mile) and multiplies distance back in afterwards. Distance stops being something the trees have to rediscover in every branch and becomes an offset, which frees the whole model to spend its capacity on the part that is genuinely hard.

The column that looks like a price and isn't

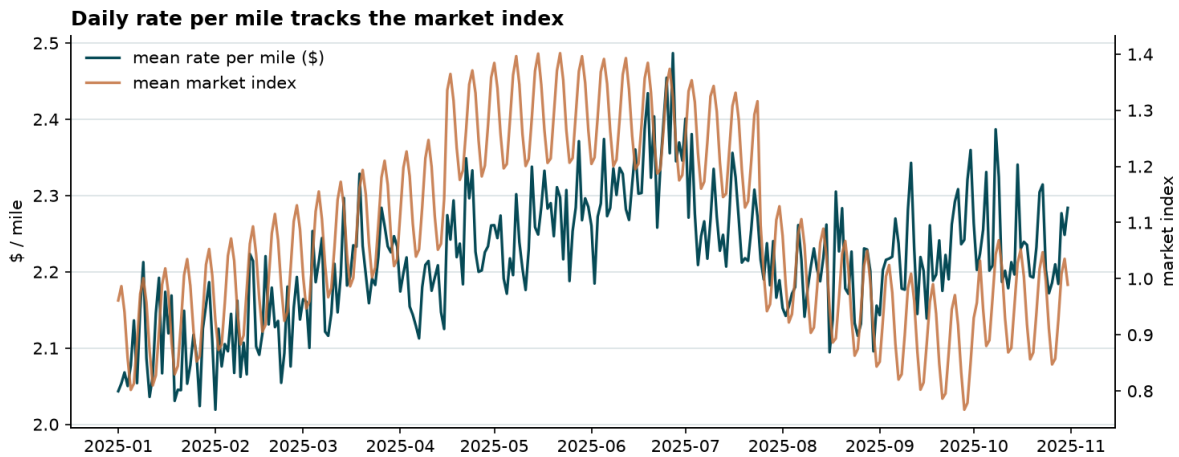
quote_signal is the trap in this dataset. It is named like a price quote, and its mean sits right next to the mean rate per mile, so it reads at a glance as though someone has handed you most of the answer. Its correlation with the realised rate per mile is 0.05. It is noise dressed up in a suggestive name, and it is dropped everywhere in this pipeline.

The lesson generalises past this dataset: a column's name is a claim made by whoever built the table, not evidence. It costs one correlation to check and it can cost a whole model not to.

The column that works, but only from a distance

market_index is the mirror image. Row by row it correlates with rate per mile at 0.08, which on its own would look like another decoy. Average it to a day and the correlation jumps to 0.58. It is a daily market reading published per load with per-row noise on top, and the noise is large enough to bury it if the column is fed in raw.

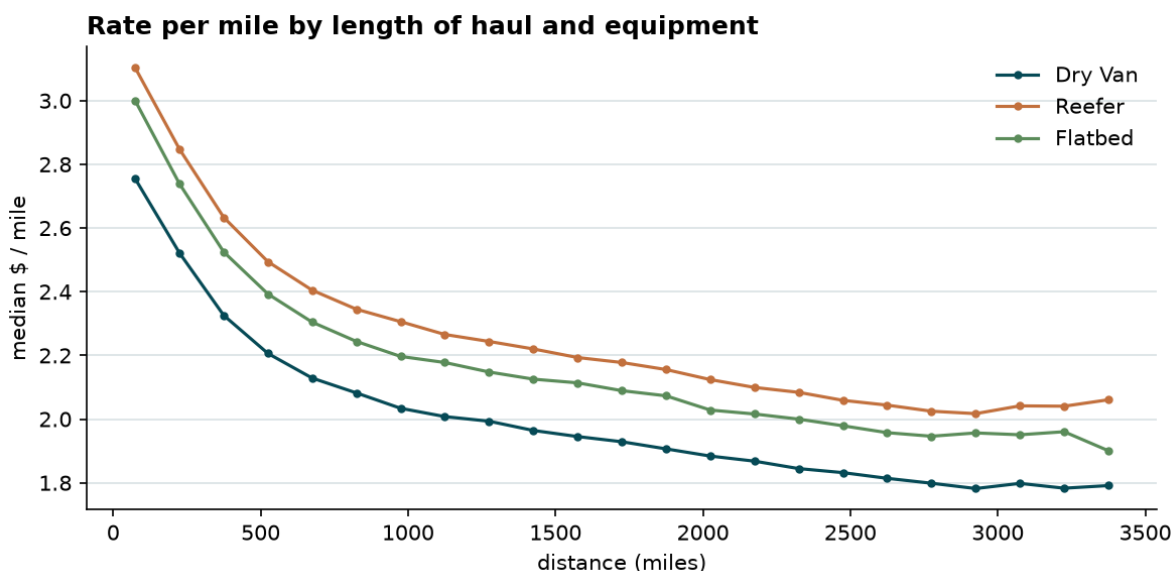
The model therefore builds a market calendar - one row per day, carrying that day's mean, seven- and twenty-eight-day rolling means, and the gap between the two as a crude momentum term. That calendar does double duty later, because it is also what lets the December chart rows be scored at all.



Daily mean rate per mile against the daily mean market index. Invisible row by row, unmistakable once aggregated.

Two more things worth knowing

- **Rate per mile falls with the length of the haul**, from roughly \$2.74 a mile under three hundred miles to \$1.90 over twenty-five hundred, and the three equipment types sit on visibly different curves - Dry Van at 2.05 a mile at the median, Flatbed at 2.22, Reefer at 2.31.
- **8 cities appear in the prediction set that never appear in training** (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego). Any model leaning on a raw city identifier would have had nothing to say about them. Geography therefore enters through coordinates, great-circle distance and bearing, and every lane-level encoding carries a fallback and a flag saying whether it was ever seen.



Rate per mile by length of haul and equipment type.

Data quality, and two problems I deliberately left alone

Three defects are real and were repaired. Two more look like defects and were not, which took longer to establish than the repairs did.

Finding	Rows	What was done
Negative weight, correct magnitude	292	Absolute value - a sign error, not a measurement
Missing weight	300	Left missing and flagged; the boosters split on missing natively
Missing market_index	374	Filled from that day's market calendar and flagged
distance long against great-circle	214	Left alone - see below
Extreme rate-per-mile outliers	~1%	Left in - absorbed by a robust loss instead

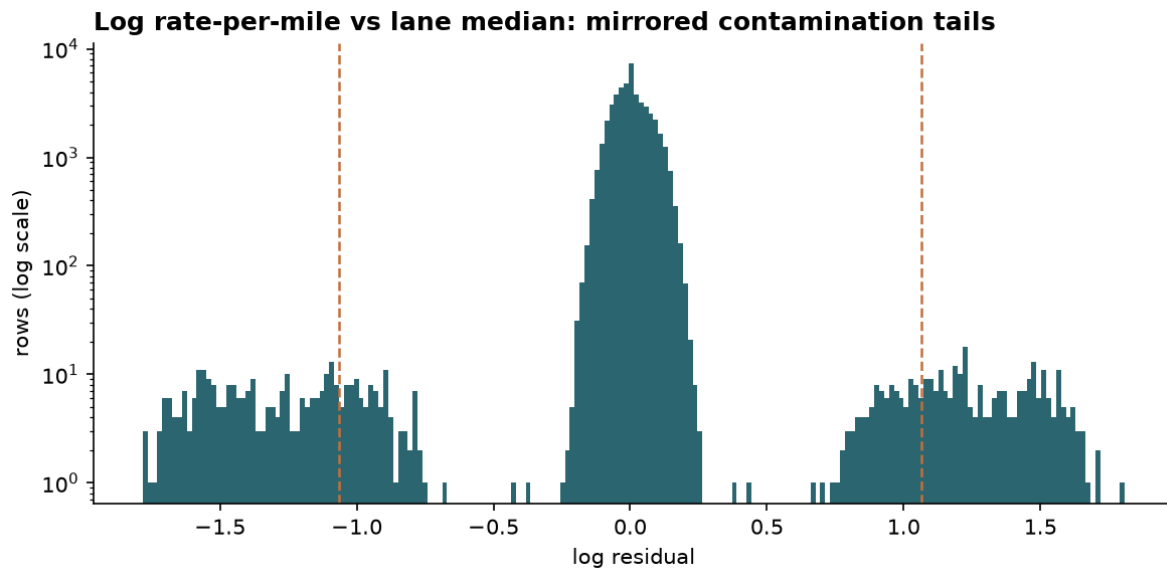
Why the suspicious distances were not repaired

Two hundred and fourteen rows carry a distance far longer than the great-circle distance between their endpoints - one by a factor of nine. The obvious move is to overwrite them with the lane median. The obvious move is wrong. Grouping the ratio by lane shows a standard deviation of 0.02 *within* each lane: the discrepancy is a fixed property of the city pair, not a per-row accident. The coordinates in this dataset are synthetic and shifted from real geography by varying amounts, so for a handful of short-haul pairs the fake coordinates land implausibly close together while the mileage stays real. Repairing those rows would have corrupted 214 correct records to fix nothing.

Why the outliers were kept

Measured against each lane's own median, the log rate-per-mile residuals have tails that are almost perfectly mirrored: -1.07 at the half-percentile against +1.06 at the ninety-nine-and-a-half. Real freight pricing is not symmetric in log space - hot markets spike upward far more readily than they collapse. That symmetry is the fingerprint of an injected multiplicative shock, roughly one row in a hundred multiplied or divided by about three.

Deleting them was tempting and would have been a mistake, because the upper tail is not purely synthetic: it clusters in June, the peak of the market in this data, so some of those rows are genuine surge pricing that the model should learn from. Rather than choose which extreme rows to believe, the primary learner optimises absolute error. That estimates the conditional median instead of the mean, and a median simply does not move when one row in a hundred is multiplied by three. The contamination is handled by the objective rather than by a deletion rule that would have thrown away real signal.



Log rate per mile against the lane median. Dashed lines mark the 0.5th and 99.5th percentiles - note how nearly they mirror each other.

How the data was split, and why it matters more than usual

A random hold-out would have been the wrong instrument here, and not by a small margin. Splitting rows at random lets the model train on the same weeks it is scored on, so it never has to survive the thing the real task demands: a two-month step into dates it has never seen.

Validation is therefore rolling-origin. Three folds, each hiding a contiguous two-month future block at exactly the horizon the submission faces. Within each fold, base learners are fitted on the earliest stretch, the dollar calibration factor is fitted on the most recent two months before the hidden block, and only then is the hidden block scored.

Fold	Fitted on	Calibration window	Scored on	Rows scored
1	9,255 rows	9,855 rows	2025-05-01 to 2025-06-30	9,696
2	19,110 rows	9,696 rows	2025-07-01 to 2025-08-31	9,671
3	28,806 rows	9,671 rows	2025-09-01 to 2025-10-31	9,523

- Nothing inside a scored block reaches the model. Base learners, lane encodings and the calibration factor are all fitted strictly on earlier dates.
- Lane encodings are computed out of fold, so a lane's own labels never leak into its own features. There is a self-check asserting exactly this, because it is the kind of leak that quietly flatters every number in a report.
- No raw time index is used as a feature anywhere. A decision tree cannot split past the largest value it saw in training, so a date index would silently clamp on every graded row. Seasonality enters only through cyclical day-of-year and month terms, distance to the nearest holiday, and the market calendar - all of which are defined for dates in the future.

What a random split would have told me

Worth quantifying, because it is the number a less careful write-up would have reported. The same model, the same features, scored on a random 80/20 split of the same rows, reports \$84.77 mean absolute error. Scored on time-based folds it reports \$147.05. The random split is 42% optimistic - it would have promised roughly half the error the model can actually deliver on the task as posed. Both numbers are in the repository; only one of them means anything.

The model

Five learners are fitted over one shared design matrix so they can be compared honestly. The shipped predictor is a fixed, equal-weight average of the two absolute-error boosters. Nothing about that weighting is fitted, and the next section is the story of why.

Learner	Role	Weight
hgb_deep	Gradient boosting, absolute-error loss, deeper trees	0.50
hgb_mae	Gradient boosting, absolute-error loss	0.50

The features fall into five groups. Geography: log distance, great-circle distance, the ratio between them, bearing, endpoint and midpoint coordinates. The load itself: weight, weight per mile, a missing flag, equipment indicators and a distance band. Calendar: day of week, cyclical day-of-year and month terms, and distance to the nearest holiday. Market: the daily calendar described earlier. And lane history: shrunk out-of-fold median log rate-per-mile encodings for the lane, the origin, the destination, the corridor, origin-by-equipment and equipment-by-distance-band, each carrying a support count and a flag for whether it was ever observed.

Predictions come back from log space through a single multiplicative calibration factor (1.0300) fitted on held-out data. A log-space fit is biased in dollar space, and the textbook smearing correction targets the mean rather than the metric in play, so the factor is simply searched for against dollar error instead of assumed. It is the one quantity the held-out block still determines.

Three ensembles that did not work

This is the part of the project that consumed the most time and produced the least code, and it is worth writing down properly, because each attempt failed for a different and instructive reason.

Attempt one: non-negative least squares in log space

The natural first move. Fit non-negative weights over the base predictions, constrained to sum to one. It produced a blend scoring \$188 mean absolute error when its own best member scored \$166. A combination that loses to one of its ingredients is not a subtle bug, and the cause was plain once stated: least squares in log space optimises squared error on log rate per mile, while the model is graded on absolute error in dollars. Those are different objectives, and the weights were dutifully optimising the wrong one.

Attempt two: greedy selection against the real metric

Replacing least squares with Caruana-style greedy ensemble selection, scored directly on dollar mean absolute error, fixed that immediately - the blend went from \$188 to \$152 and drew level with its best member. Then the full-data run exposed something worse. On the earliest fold the blend scored \$406 where its best member scored \$118. Inspecting the weights showed it had handed 74% of the ensemble to the ridge model, which went on to post \$498 on the very block it was chosen to predict.

Attempt three: bagging the selection

Greedy selection is known to overfit small selection sets, and the published remedy is to bag it - run the selection over bootstrap subsamples and average the resulting weights. That was implemented, and it moved ridge's weight from 0.744 to 0.744. A fix that changes nothing is information: the failure was not sampling noise, so no amount of resampling the same window would ever have addressed it.

What was actually wrong

The weights were being learned from one set of models and applied to a different one. Base learners are fitted on the pre-calibration window, the weights are learned from those fitted models, and then the learners are refitted on the pre-calibration window plus the calibration window before anything is predicted - so that the shipped model has seen the most recent weeks. On the earliest fold that refit doubles the training data, from 9,255 rows to 19,110. Gradient boosting improves sharply when its data doubles; ridge regression barely moves. The ranking the weights encoded was obsolete by the time they were used.

No weighting scheme can repair that, because the problem is not which weights were chosen - it is that they were chosen for a lineup that no longer exists.

With the cause understood, the evidence was unambiguous. Across all three folds, fitted blending never once beat the best single member: it tied twice and lost catastrophically once. A mechanism whose best case is a tie and whose worst case costs \$250 of mean absolute error does not belong in a shipped model. The fitting was removed.

What ships instead is an equal-weight average of the two absolute-error boosters. There are no fitted parameters, so the mismatch cannot recur by construction. Averaging two near-equivalent learners is a free reduction in variance, and it sidesteps the awkward question of why one was picked over another when cross-validated MAE separates them by ten cents. All five learners are still fitted and reported, because the comparison is evidence; only two of them carry any weight at prediction time.

Results

Averaged across the three rolling-origin folds. The shipped model reaches \$147.05 mean absolute error at 6.23% MAPE, against \$209.79 for the strongest naive rate-per-mile rule - an error reduction of 30%.

Model	MAE	RMSE	MAPE	Med APE	R ²
Shipped model: equal-weight average of the two robust boosters	\$147.05	\$637.68	6.23%	3.66%	0.824
Gradient boosting, absolute-error loss, deeper trees	\$147.18	\$637.83	6.23%	3.67%	0.824
Gradient boosting, absolute-error loss	\$147.21	\$637.67	6.24%	3.66%	0.824
Random forest	\$165.28	\$645.60	7.14%	4.38%	0.820
Gradient boosting, squared-error loss	\$189.56	\$656.94	8.06%	5.05%	0.813
Baseline: lane median rate per mile x distance	\$209.79	\$671.89	8.73%	5.67%	0.805
Baseline: global median rate per mile x distance	\$249.25	\$676.94	11.56%	8.04%	0.802
Ridge regression	\$412.60	\$786.12	17.36%	15.31%	0.730

The row worth dwelling on is **hgb_mse**. It shares every feature and every hyperparameter with **hgb_mae** and differs in exactly one respect - it minimises squared error instead of absolute error - so the gap between them (\$147.21 against \$189.56) is a clean measurement of what the injected shocks cost a model that tries to fit them. It is the empirical version of the argument made earlier from the shape of the residual tails.

It is also worth noticing how far the ridge row sits from the rest. A linear model on these features degrades badly across a two-month horizon, which is precisely the property that made it so dangerous when a fitted blend was free to select it on a short window.

Where the error actually sits

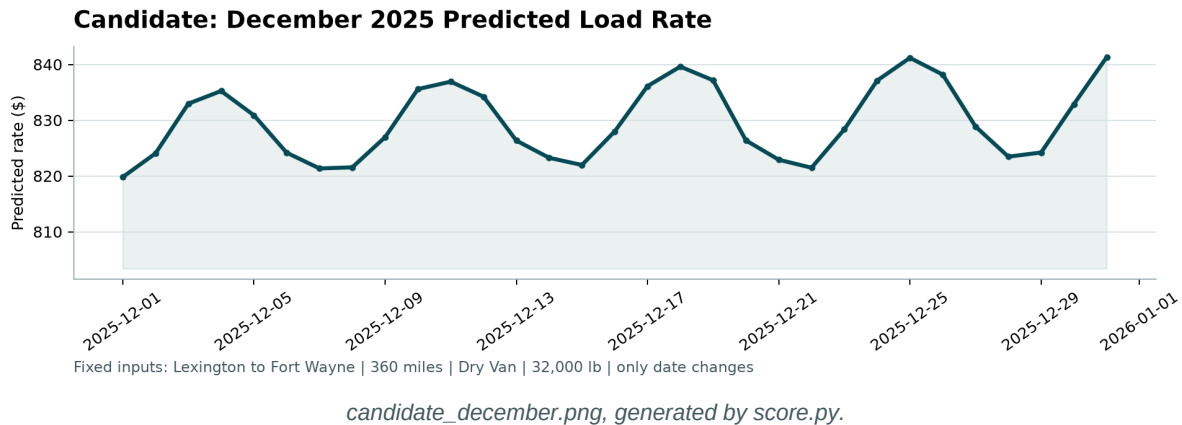
Headline numbers hide structure, so here is the most recent fold broken out by equipment and by length of haul.

Equipment	MAE	MAPE
Dry Van	\$144.71	6.03%
Flatbed	\$157.44	7.09%
Reefer	\$173.19	7.04%

Distance band (mi)	MAE	MAPE
0-300	\$31.80	6.65%
300-600	\$75.29	6.58%
600-1000	\$106.51	6.54%
1000-1500	\$169.84	6.21%
1500-2500	\$267.58	6.71%
2500+	\$315.62	5.58%

The fixed December chart

The chart below is produced by the provided scorer from the completed December input file: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb, every day of December 2025, with the date as the only thing that changes.



Those rows are a quiet test of whether the pipeline was built properly. They arrive with six columns and no coordinates, no market index and no quote signal - so a model that had bound itself to the training schema would have had nothing to score them with. Here they pass through the same cleaning and feature code as everything else: coordinates are recovered from the city table, market state from the daily calendar, both assembled from feature columns of the supplied files and never from labels.

That is also why the curve has shape rather than sitting flat. Every physical input is constant across the 31 rows; the only things moving are day of week, the cyclical calendar terms, proximity to Christmas and New Year, and the market calendar. If the feature engineering had failed, this chart would be a horizontal line, and it is the fastest visual check in the whole project.

Limitations, honestly

- **The first fold is thin.** It fits on only 9,255 rows against 28,806 for the last, so it is a harder test than the shipped model faces and it drags the averages up. It was kept rather than dropped, because it is also the fold that exposed the ensembling bug, and a validation scheme that only ever confirms the model is not doing its job.
- **Roughly one row in a hundred is unpredictable by construction.** The injected multiplicative shocks are not a function of any feature, so they set a floor on achievable error that no amount of modelling will get under. This is most visible in RMSE, which stays high while median absolute percentage error stays low.
- **The market calendar is descriptive, not predictive.** It uses the market index supplied for the prediction rows themselves, which is legitimate here because that column is a given input rather than a label. A production system would need to forecast it, and that forecast error would be additional.

- **One scalar for calibration.** The log-to-dollar correction is a single multiplier applied uniformly. Per-segment calibration - by equipment, or by distance band - would probably squeeze out a little more, but it adds fitted parameters on a small held-out window, which is exactly the pattern that already went wrong once in this project.

Reproducing all of it

Everything in this document is generated from the artefacts the pipeline writes; no number was typed in by hand. From a clean checkout:

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python src/selfcheck.py # leak, fallback and December-path assertions
python src/eda.py # data-quality report and figures
python src/validate.py # rolling-origin validation (~30 min)
python src/train.py # fits and saves the shipped model
python src/predict.py # writes both submission files
python score.py --predictions validation_predictions.csv \
--december-predictions december_chart_inputs.csv
python src/report_pdf.py # regenerates this document
```

The repository keeps its reasoning next to its code: the rejected ensembling schemes are documented in `models.ensemble_weights`, the cleaning decisions that were *declined* are documented in `data.clean`, and `src/selfcheck.py` holds runnable assertions for the failures that would otherwise be silent - an encoder that leaks, an unseen city that returns a null, a December row that cannot be scored.